Epicenter Pipeline: QA Walkthrough Script

As of 2026-11-05. Shareable copy: https://claude.ai/artifact/1nLmFSK4HBPZyeazGCYBRb

This is the narration for the screen-recorded walkthrough that teaches QA how to test the Epicenter **pipeline** feature: project setup, developer layout, GitHub sync and npm build. Spoken lines are plain text. *On screen* cues in italics say what should be visible while you say them. Reference screenshots are in `notes/screenshots/`.

Everything is done in the Epicenter admin UI and on GitHub. No files are uploaded by hand and no pipeline config files are uploaded; the pipeline is assembled in the UI.

Before you record

- Two browser tabs: the Epicenter admin UI on the test server, and GitHub.
- You are logged in to GitHub with an account that can fork a repository and add a deploy key.
- You are logged in to Epicenter with an account you can create projects in, and there is no project called `qa-pipeline-example` in it yet.
- The example repository is https://github.com/forio/forio-pipeline-example.

Running time is about ten minutes. The build itself takes about half a minute.

---

1. Introduction

> On screen: the Epicenter admin home page.

Today I'm going to show you how to use the Epicenter Pipeline. This is a new feature of Epicenter that lets you pull source code from GitHub, build it with npm, and serve the result as your sim. It's essentially Epicenter acting as its own Jenkins: a small continuous integration system built right into the platform.

I've put together an example project called forio-pipeline-example that you can fork and use to test this out. We'll get to GitHub in a few minutes. First I want to create the project on Epicenter, because there are a couple of settings we have to change before the pipeline is even available.

2. Create the project

> On screen: your account page, then the new project form.

I'm logged in to Epicenter and I'm in my account. I'll create a new project. Give it a simple name. I'll call this one qa-pipeline-example. For project access, pick Password Protected, which is what a typical sim uses. And create the project.

3. Project Files in the standard layout

> On screen: Project Files.

Let's look at the project files first. By default a project has two main sections: a model folder and an interface folder. You put your model in one, your web files in the other, and Epicenter serves them straight from there. That's the standard layout, and it's fine for a sim you upload by hand.

But a pipeline needs one place to put source code and a separate place to put the built output. So we need to switch this project over to the developer layout.

4. Change the settings

> On screen: Settings, scrolled to Advanced Settings.

Go to Settings, and scroll down to Advanced Settings. I want to change two things here.

The first is Web Access. I'm going to set that to "Allow access to all URLs by default". This is pretty standard for a simulation, because the sim handles its own login. If you skip this one, Epicenter will try to send visitors to a login page that this sim doesn't have, and the sim will look broken.

The second one is the key setting: Directory Layout. Change it from Simple to Developer. Then save the project settings.

5. Project Files in the developer layout

> On screen: Project Files again, then the sidebar.

Now go back to Project Files, and you can see the layout has changed. Instead of model and interface at the top, we have a source section and a target section. Source has model, interface, proxy and data folders. Target has the same four.

Here's the idea. Our source code from GitHub is going to land in source. The build process is going to compile it and put the result in target. And the sim is served out of target. That's the whole job of the pipeline: get code into source, and turn source into target.

And notice there's a new item in the sidebar now: Pipeline. It only appears in the developer layout. We'll come back to it. Before we can use it, we have to tell Epicenter where the code lives.

6. GitHub: the example project and your fork

> On screen: the GitHub tab, at forio/forio-pipeline-example.

Let's go over to GitHub. You'll need a GitHub account for this, so create one if you don't already have one. This is the forio-pipeline-example repository. It's a very simple sim based on dev-base-build: it lets you log in, and it has a small Excel model.

Look at the folder layout for a second. There's an interface folder, which is the npm project, the React source code with its package.json. And there's a model folder with the Excel model. Those two names match the folders inside source back on Epicenter, and that's not a coincidence. The top level of this repository is going to become the source folder on the platform.

You'll want your own copy of this, so you can add a deploy key to it and change things without touching the original. Click Fork. It shows your GitHub username as the owner. Keep the repository name as it is, and create the fork. Now I have my own copy under my account.

7. Connect Epicenter to GitHub

> On screen: Epicenter Settings, the Git Integration section. Keep the fork open in the other tab.

Back to Epicenter. Go to Settings, and near the top there's a section for Git Integration. Click that. I need to fill in two things: where the repository is, and a key so that Epicenter is allowed to read it.

First, the repository URL. This is not the address in your browser bar. On GitHub, on your fork, click the green Code button and choose SSH. The address starts with git at github dot com. Copy that, come back to Epicenter, and paste it into the repository URL field.

You can set a passphrase if you like. I'm going to leave it blank.

Now the key. Epicenter has already generated a deploy key for this project. It's right here, with instructions next to it. This is the public half of the key; GitHub uses it to recognize Epicenter. So I copy it, go back to my fork on GitHub, open the repository's Settings, scroll down to Deploy keys, and click Add deploy key. Give it a simple title, something like "epicenter pipeline". Paste the key in. Leave "Allow write access" unchecked, because Epicenter only needs to read. Add the key.

So now both sides know this key, and Epicenter is able to pull files from GitHub.

> On screen: back on Epicenter, the Initialize Repository button and its warning.

Back on Epicenter I'll click Initialize Repository. Read the warning: this is going to delete any files currently in the project and replace them with the contents of the repository. That's fine, there's nothing in here yet. Go ahead.

8. The source files are in

> On screen: Project Files, inside source.

Let's check. Project Files, open source, and now I have the model files and the interface files. These came straight from GitHub.

But these are source files. They aren't ready to run. If I clicked Run Project right now there'd be nothing there, because target is still empty. What I have to do is run npm to build the interface, and copy the model across. That is exactly what the pipeline does.

9. Build the pipeline in the UI

> On screen: Pipeline in the sidebar, then New Pipeline (screenshots 1 and 2).

Click Pipeline in the sidebar. This is where you define a job, the equivalent of a Jenkins job. Click New Pipeline.

A pipeline is just a list of operations that run in order, top to bottom. Start by giving it a name over on the right. I'll call it git-and-build. The name becomes the config name on the back end, so keep it simple, no spaces.

> On screen: the Add Operation dialog (screenshot 3).

Now click Add Operation. There are four kinds. Clean clears out the target directory. Git Synchronize pulls from the repository. NPM runs an npm command in a container. And Copy copies files between locations. We're going to use three of them today.

First, Git Synchronize. This matches the project's copy of the code to the remote branch. We already set up the git integration, so it knows where to pull from. Branch is main.

Next, NPM. This runs a Node.js command inside a Docker container. For this example we need two npm steps. Which commands you need will depend on the codebase, but this is the standard pattern. The first command is npm ci, which installs the dependencies. Directory is Interface, because that's where package.json lives. Node version is 24. The timeout is in minutes; ten is plenty.

Add another NPM operation with the same settings, and this time the command is npm run build. That compiles the TypeScript and bundles everything into the files the browser loads, and it writes them straight into target.

Last one: Copy. We need the model to go from source to target as well. Location is Model. Leave From and To blank, which means the whole model folder, root to root.

> On screen: the four operations listed in order.

So, reading top to bottom: sync from git, install dependencies, build the interface, copy the model. Save the pipeline.

10. Run it and read the results

> On screen: Execute, then the run page as it progresses (screenshot 4, top and bottom).

Now click Execute. It takes you to the run page and you can watch it go step by step.

Git Synchronize finishes almost instantly. It shows the commit it synced to and the branch. Then the first npm step starts, npm ci. This is the slow one, because it's downloading packages, so give it twenty or thirty seconds. Each operation has an Output section you can expand to see the actual npm output.

Now the build step. Done, it created the HTML and the JavaScript. And finally the copy. At the top it says Succeeded, with the duration, who started it, and when. And at the bottom there's a Raw Log with the full detail, if you ever need to work out why a step failed.

> On screen: Back to History, then the Pipeline list.

Click Back to History. Every run of this pipeline is listed here, whether it succeeded or failed. And if you go back to the Pipeline page, the pipeline is in the list with buttons to run it again, duplicate it, or delete it. You can create as many pipelines as you want; for example, one that only syncs from git and one that does the full build.

11. Target, and the running sim

> On screen: Project Files, target, then interface and model inside it. Then Run Project.

Let's go back to Project Files. Source is still the same raw files from GitHub. But now look at target. Target interface has the built files: index.html and an assets folder with the compiled JavaScript and CSS. Target model has the Excel model. Nothing in here was uploaded by hand. It is all pipeline output.

And now if I click Run Project, there's the sim. I can log in and start playing.

(The project is password protected, so you need a workshop with at least one user in it to log in. If you don't have one, create it under Workshops before you record this part.)

So that's how the pipeline works. Think of it as a CI system like Jenkins, built into Epicenter. Source comes in from GitHub, npm builds it, the result is served from target, and every run is logged with who started it and when. Thanks so much.

What QA should verify

Each of these is a pass/fail check while following the script.

- ☐ A new project starts in the standard layout: Project Files shows model and interface.
- ☐ After saving Web Access "Allow access to all URLs by default" and Directory Layout "Developer", Project Files shows source and target, each with data, interface, model and proxy, and Pipeline appears in the sidebar.
- ☐ The fork exists under your GitHub account.
- ☐ Git Integration accepts the SSH repository URL from GitHub's Code button and shows a deploy key to copy.
- ☐ The deploy key is listed on the fork under Settings, Deploy keys, as read-only.
- ☐ Initialize Repository warns that existing files will be deleted, then source contains the repository (interface, model, pipeline, tests, README).
- ☐ New Pipeline accepts a name and the four operations: Git Synchronize (branch main), NPM npm ci, NPM npm run build (both Directory Interface, Node v24), Copy (Location Model).
- ☐ Execute runs the operations top to bottom; each shows Completed and the run shows Succeeded with Started By, Started, Duration, per-step Output and a Raw Log.
- ☐ History lists the run; the Pipeline page lists the pipeline with run, duplicate and delete actions.
- ☐ target/interface contains index.html and assets; target/model contains model.xlsx.
- ☐ Run Project opens the sim, and a workshop user can log in.

If something goes wrong

- **Git Synchronize fails to authenticate.** The deploy key was added to the original forio repository instead of your fork, or not added at all. Add it under the fork's Settings, Deploy keys, and run again.
- **Git Synchronize fails with "Illegal character in scheme name".** Enter the URL in the form `ssh://git@github.com/<your-user>/forio-pipeline-example.git`.
- **npm ci fails.** The fork's `interface/package-lock.json` is missing or out of date. Commit a matching lockfile and run again.
- **Later operations show Skipped.** Expected: the pipeline stops at the first failure. Fix that step and run again.
- **Run Project shows 404.** Nothing has been built yet, or the build wrote somewhere else. Check target/interface for index.html and re-run the pipeline.
- **Run Project redirects to login.html.** Web Access is not set to "Allow access to all URLs by default". Change it in Settings, Advanced Settings.

Clean-up after the session

- Delete the project `qa-pipeline-example`, or keep it as a regression fixture.
- Remove the deploy key from your fork.
- Delete any workshop users created only for the walkthrough.